



Chapter 10: Multithreading



Multithreading

- ❑ Processes and threads
- ❑ Multithreading
- ❑ Control of threads in Java
- ❑ Sync, deadlocks, and how to avoid them
- ❑ Multi-threaded applications—producer vs. consumer mode
- ❑ Task vs. thread pool

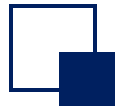


Processes and threads

What is a process?

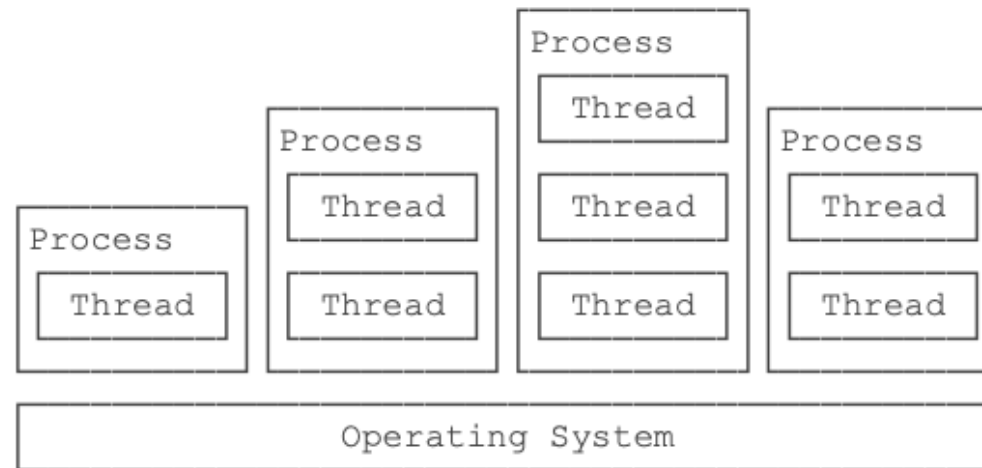
❑ Process: An instance of a running program

- Private space, isolated from each other
- Each process have the resources of the entire computer
- Memory is not shared between multiple processes
- Processes collaborate through messaging
- In general: process == program == application
 - ✓ But an app can also contain multiple processes



What is a thread?

- ❑ Thread: A single sequence of control flows in a process
 - The smallest unit of operational scheduling that an operating system is capable of
 - Included in the process is the actual operating unit of the process
 - A process can contain (concurrently) multiple threads
 - A process contains at least one thread
 - Memory is shared between multiple threads





Threads and processes

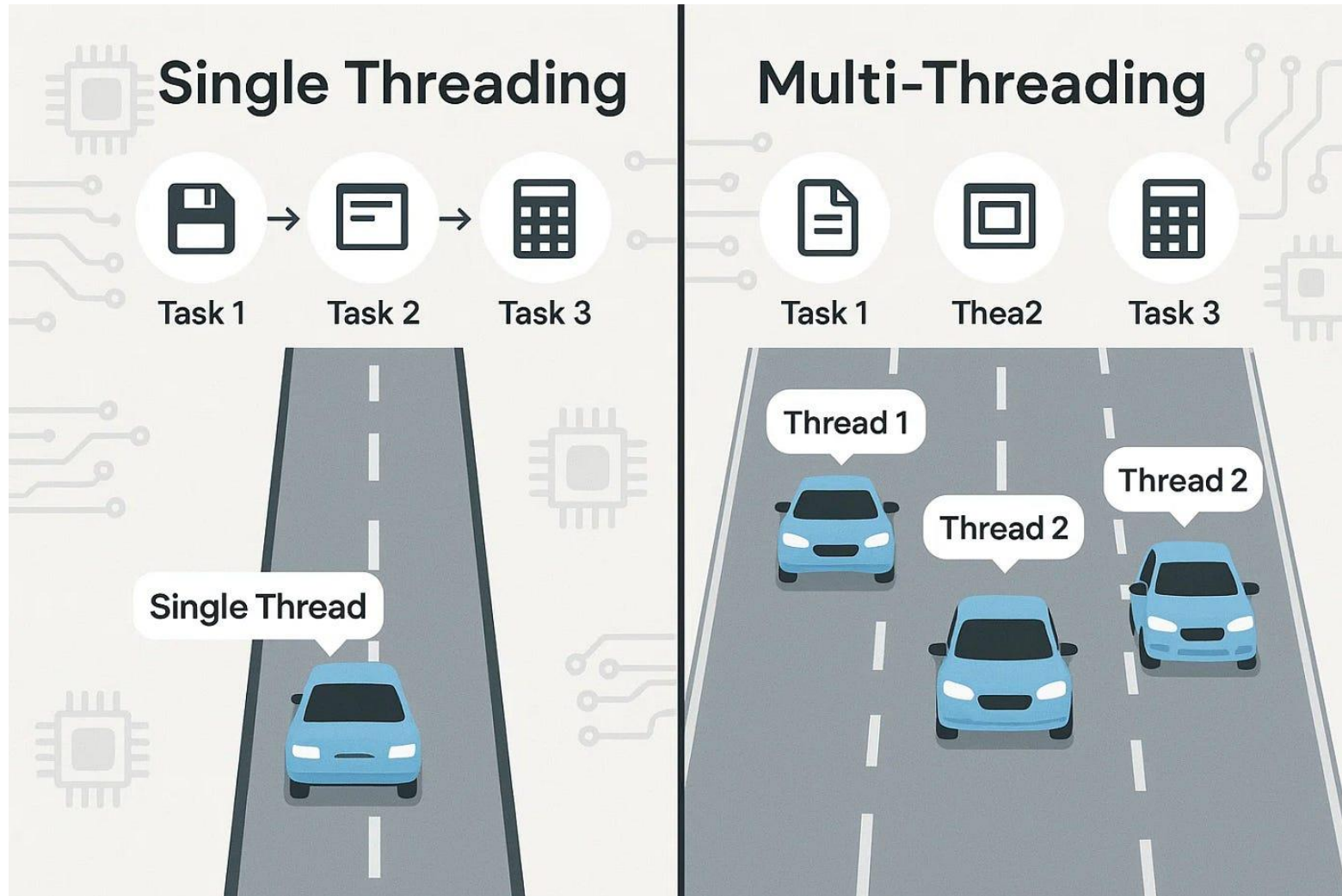
Process	Thread
Heavyweight	Lightweight
An app can contain multiple processes	A process can contain multiple threads
Memory is not shared between multiple processes	Multiple threads of a process share memory
The process appears as a virtual machine	Threads behave as virtual CPUs

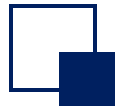


Multithreading



"Multithreading" in life



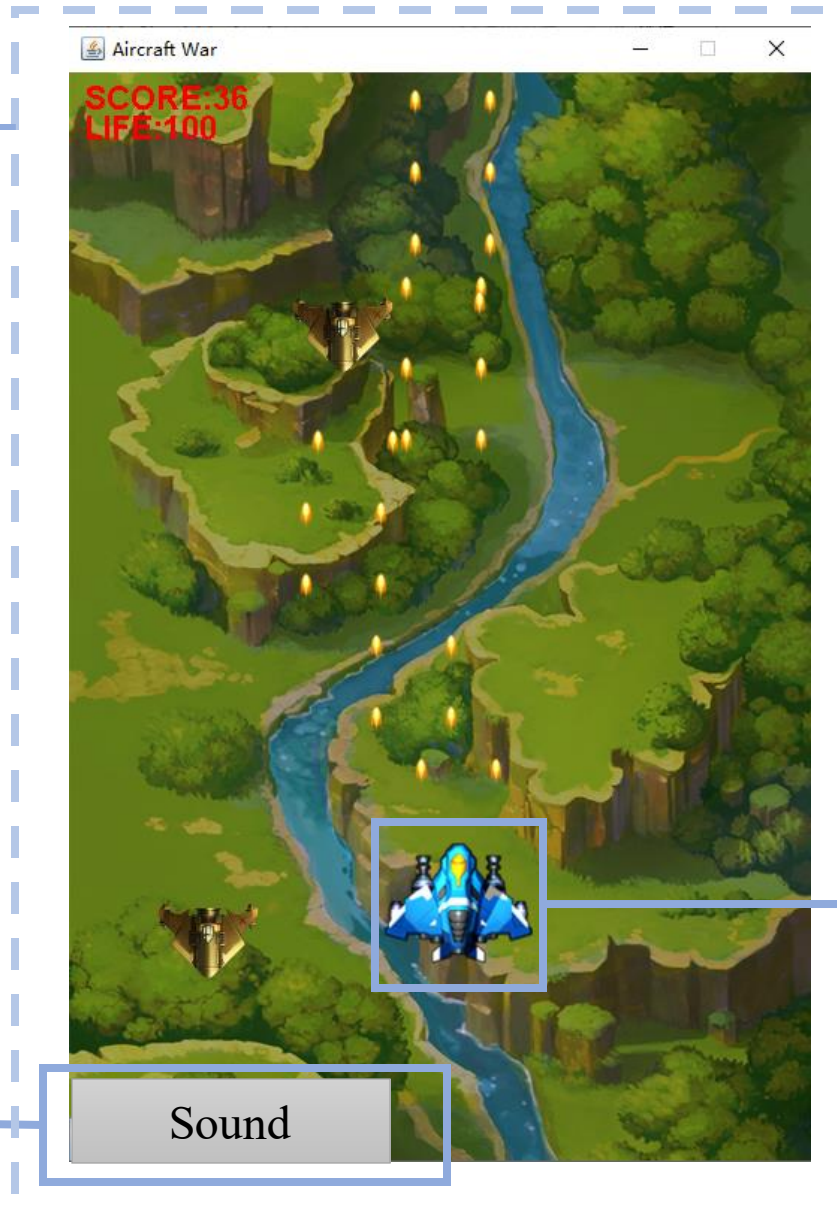


Multithreading in aircraft war

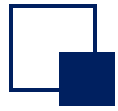
Aircraft War
Application Process

Game Control Main Thread
Game logic threads

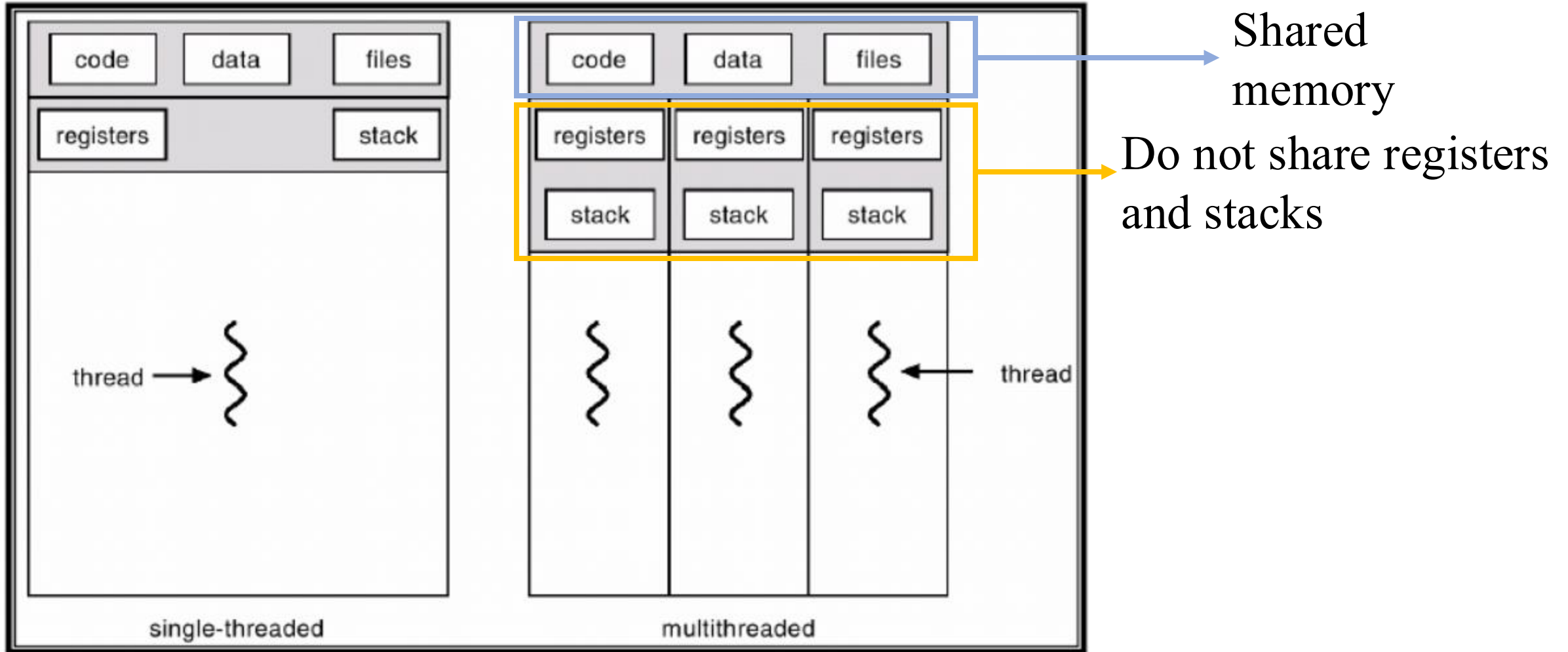
Sound control threads



Threads for
keyboard listening



Multithreading

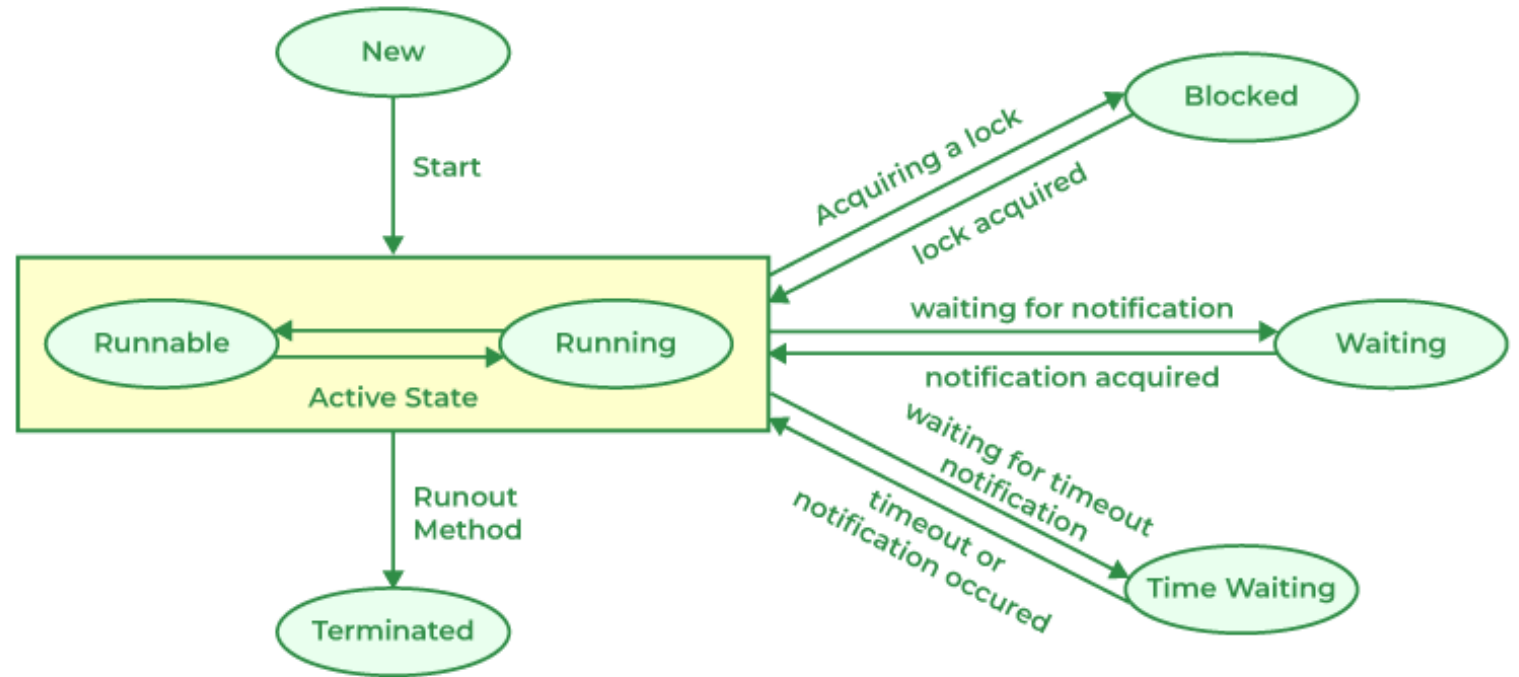


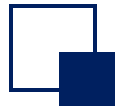


Control of threads in Java

The state of the thread

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated



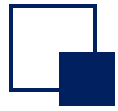


Create a thread

- Two ways
 - Create a subclass of the Thread class
 - Implement the Runnable interface
- Call the start method
 - Start the thread, which will raise the call to the run method;
 - start method will return immediately;
 - New threads will run concurrently.
- Note: The run method cannot be called directly
 - Calling the run method directly will only execute tasks in the same thread without starting a new thread.

```
1 public class Thread1 extends Thread {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
2 public class Thread2 implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
public class Main {  
    public static void main(String[] args){  
        new Thread1().start();  
        new Thread(new Thread2()).start();  
    }  
}
```

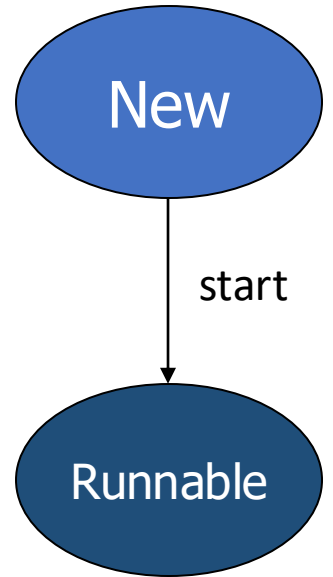
New

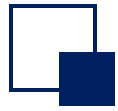


Create a thread

- Runnable is more commonly used and has the following advantages:
 - Decouple tasks from the operating mechanism to reduce overhead;
 - It is easier to achieve multi-threaded resource sharing;
 - Avoid the impact of single succession limitations.

```
public class Thread1 extends Thread {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
public class Thread2 implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
public class Main {  
    public static void main(String[] args){  
        new Thread1().start();  
        new Thread(new Thread2()).start();  
    }  
}
```

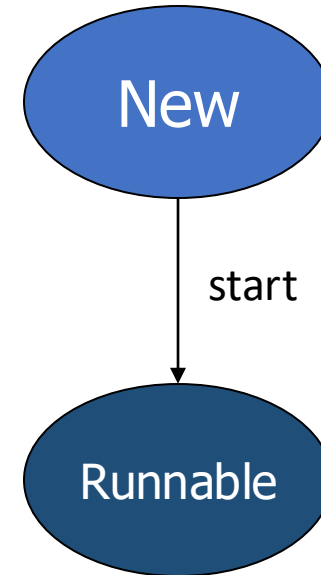


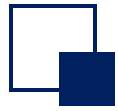


Create a thread

- The lambda syntax introduced after Java 8 is further abbreviated as:
 - Create and start a thread

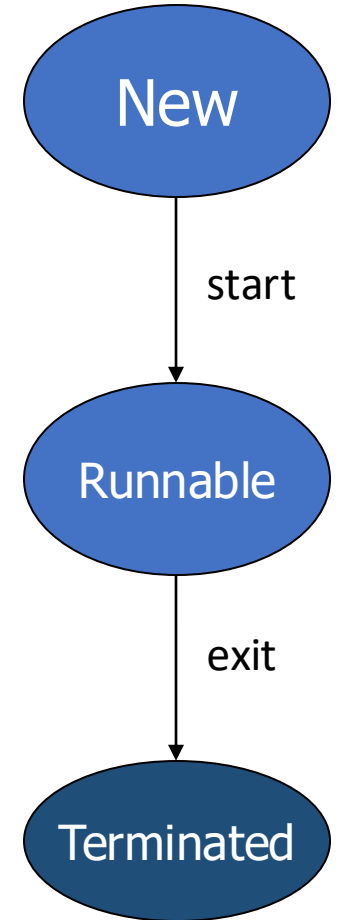
```
public static void main(String[] args) {  
    //lambda  
    Runnable r = () -> {  
        System.out.println("Thread: Runnable with Lambda Expression");  
    };  
    new Thread(r).start();  
}
```

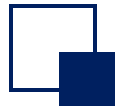




Terminate the thread

- The thread terminates for one of two reasons:
 - run method exits normally, and the thread terminates naturally;
 - Because an exception that was not caught terminated the run method, causing the thread to terminate unexpectedly.
- Note: The stop method has been deprecated
 - Calling the thread's stop method terminates the thread.
 - The method throws a ThreadDeath error object, thereby killing the thread.
 - Unsafe: The method attempts to terminate a given thread without thread interop.





The state of the thread

by the runnable state →

- Timed Waiting

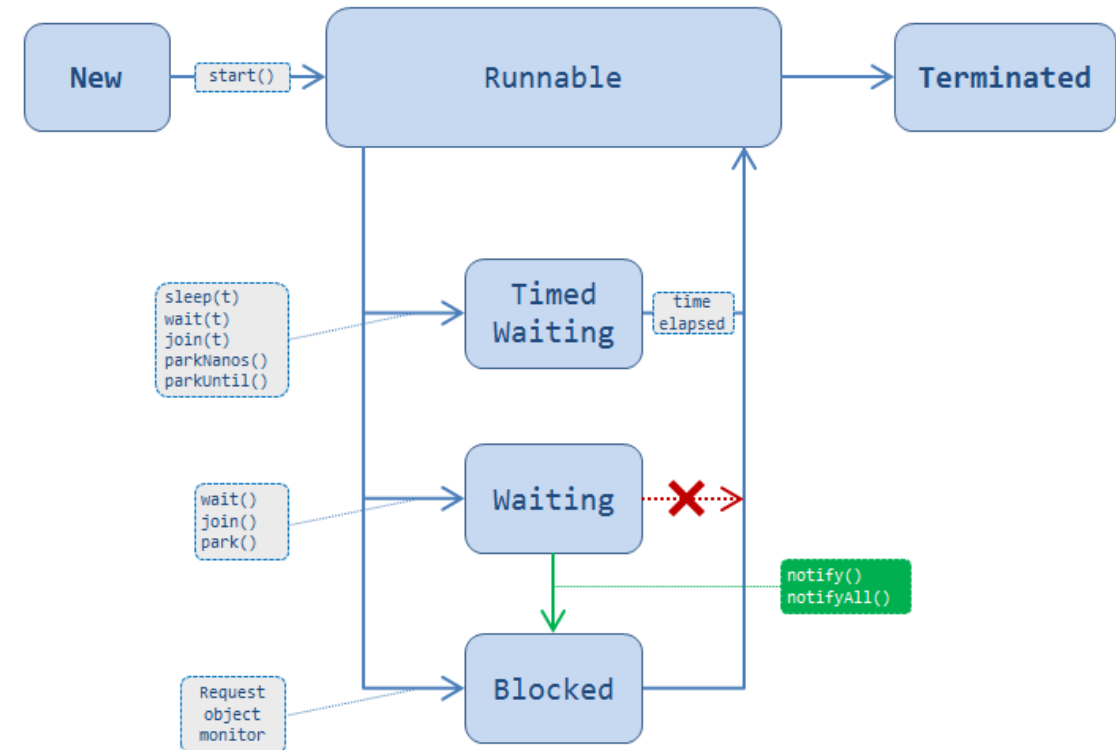
- Methods that have a timeout parameter that causes the thread to enter a timed waiting state.

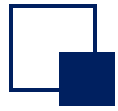
- Waiting

- When a thread waits for another thread to notify the scheduler of a condition, it enters the waiting state on its own.

- Blocked

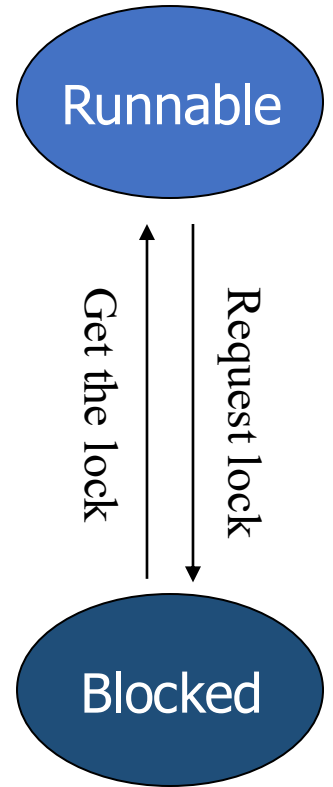
- When a thread tries to obtain an internal object lock that is held by another thread, the thread enters the blocking state.
- When all other threads release the lock, and the thread scheduler allows this thread to hold it, the thread becomes non-blocking.





Thread blocking

- When a thread enters a blocking state:
 - When a thread tries to acquire an internal object lock that is held by other threads.
- When a thread becomes non-blocking:
 - When all other threads release the lock and the thread scheduler allows the thread to hold it.
- When a thread is blocked or waiting, it is temporarily inactive
 - It doesn't run any code and consumes minimal resources. Until the thread scheduler reactivates it.





Runnable → Blocked → Runnable

```
class SharedResource {
    synchronized void access(String name) {
        System.out.println(name + " got the lock and is working...");
        try {
            Thread.sleep(2000); // hold the lock for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(name + " finished and released the lock.");
    }
}

class MyTask implements Runnable {
    private final SharedResource resource;
    private final String name;

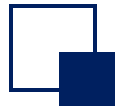
    MyTask(SharedResource resource, String name) {
        this.resource = resource;
        this.name = name;
    }

    public void run() {
        resource.access(name);
    }
}

public class BlockedExample {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        Thread t1 = new Thread(new MyTask(resource, "Thread-1"));
        Thread t2 = new Thread(new MyTask(resource, "Thread-2"));

        t1.start();
        t2.start();
    }
}
```



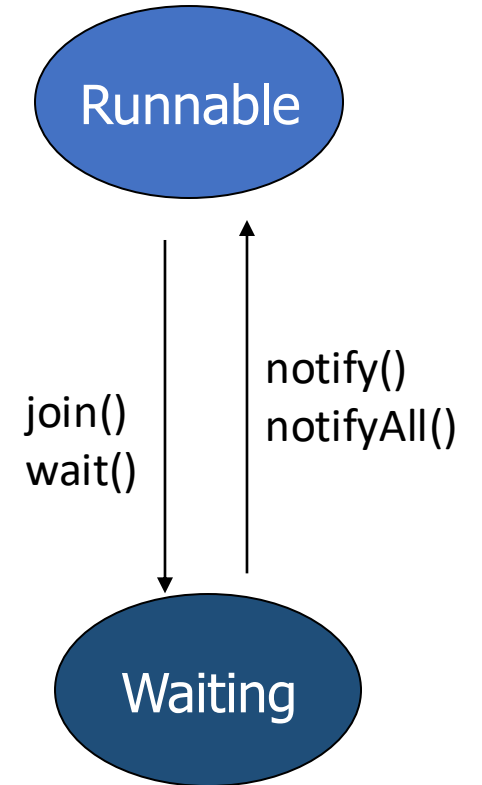
Thread waiting

- Runnable → Waiting :

- Scenario 1: The current thread object calls the `Object.wait()` method;
- Scenario 2: Other threads call the `Thread.join()` method (join is the main thread waiting for the termination of the child thread).

- Waiting → Runnable :

- Scenario 1: The waiting thread is awakened by another thread object and calls `Object.notify()` or `Object.notifyAll()`.
- Scenario 2: The thread that calls the `Thread.join()` method ends.



Runnable → Waiting → Runnable

```
class SharedTask implements Runnable {
    private boolean ready = false;

    public synchronized void waitForSignal() {
        while (!ready) {
            try {
                wait(); // wait until notified
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Resumed!");
    }

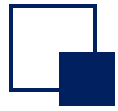
    public synchronized void sendSignal() {
        ready = true;
        notify(); // resume the waiting thread
    }

    public void run() {
        waitForSignal();
    }
}
```

```
public class WaitNotifyExample {
    public static void main(String[] args) {
        SharedTask task = new SharedTask();
        Thread thread = new Thread(task);
        thread.start();

        try {
            Thread.sleep(1000); // simulate delay
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        task.sendSignal(); // resume the waiting thread
    }
}
```



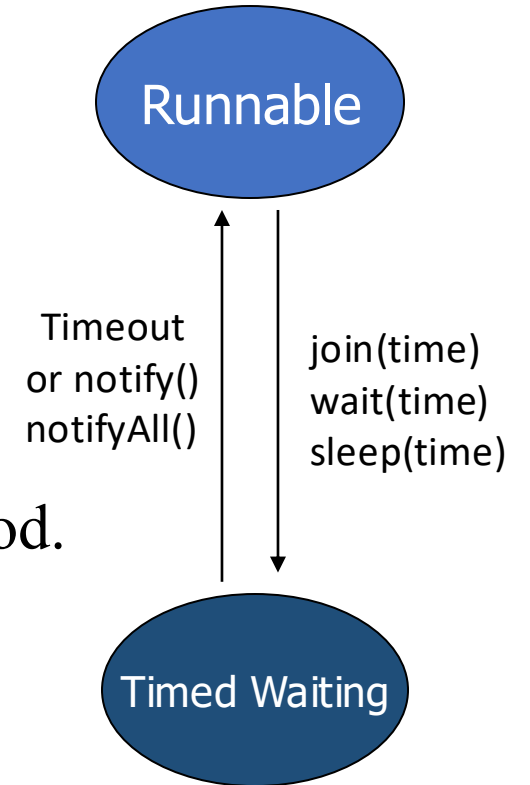
Thread timing waits

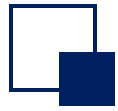
- Runnable → Timed Waiting:

- Scenario 1: The current thread object calls the `Object.wait(time)` method;
- Scenario 2: Other threads call the `Thread.join(time)` method.
- Scenario 3: The current thread calls the `Thread.sleep(time)` method.

- Timed Waiting → Runnable :

- Unlike waiting, it can return on its own at the specified time.

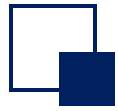




Runnable → Timed Waiting → Runnable

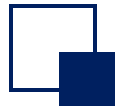
```
class TimedTask implements Runnable {
    public synchronized void run() {
        System.out.println("Task started, waiting for 2 seconds...");
        try {
            wait(2000); // timed waiting for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Task resumed after waiting!");
    }
}

public class TimedWaitExample {
    public static void main(String[] args) {
        TimedTask task = new TimedTask();
        Thread t = new Thread(task);
        t.start();
    }
}
```



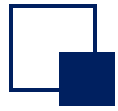
Interrupt the thread

- When the interrupt method is called on a thread, the thread's interrupt state is set
 - This is the boolean flag that every thread has (true: interrupted; false: uninterrupted).
 - Each thread should check this flag from time to time to determine if the thread is interrupted.
- When the interrupt method is called on a blocked thread
 - Blocking calls (i.e., sleep or wait calls) will be interrupted by an InterruptedException exception.
- Interrupting a thread is nothing more than getting its attention
 - The interrupted thread can decide how to respond to the interrupt.



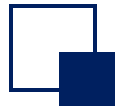
Thread priority

- Each thread has a priority
 - By default, a thread inherits the priority of its parent thread;
 - You can use the `setPriority(int newPriority)` (1~10, default value 5) method to increase or decrease the priority of any thread.
- Whenever the scheduler decides to run a new thread, it first chooses among the threads with high priority, although this will completely starve the low-priority threads
- Thread priority is highly system-dependent
 - In Oracle's Java virtual machines for Linux, thread priority is ignored.



Daemon thread

- Use the `setDaemon` method to identify the thread as a daemon thread or a user thread.
 - `thread.setDaemon(true); // true: Daemon thread`
- The sole purpose of a daemon thread is to serve other threads.
 - Example: Timing thread. It sends a "timer tick" signal to other threads at regular intervals.
- The end of the daemon thread is after the `run` method runs or the main function ends.



Summary of related methods for threads

Core methods in the Thread class

Method Name	Static?	Method description
start()	no	Let the thread start, enter the ready state, and wait for the CPU to allocate a time slice
run()	no	The method of rewriting the Runnable interface is the specific logic that the thread executes when it obtains the CPU time slice
yield()	yes	The courtesy of threads makes the thread that obtains the CPU time slice enter the ready state and compete for the time slice again
sleep(time)	yes	The thread hibernates for a fixed time, enters a blocked state, and after the hibernation time is completed, it re-competes for the time slice, and the hibernation can be interrupted
join()/join(time)	no	When the join method of the thread object is called, the caller thread enters the blocking, and waits for the thread object to be executed or reaches the specified time before resuming, and then re-competes for the time slice
isInterrupted()	no	Get the interrupt mark of the thread, true: interrupted, false: not interrupted. The break tag is not modified after the call
interrupt()	no	The method of interrupting a thread and throwing an InterruptedException exception can be interrupted, but the interrupt mark will not be modified after interrupting
interrupted()	no	Get the interrupt mark for the thread. The break mark is cleared after calling
currentThread()	yes	Get the current thread

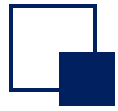
Summary of related methods for threads

Object thread-related methods

Method Name	Method description
wait()/wait(long timeout)	The thread that gets the lock enters a blocking state
notify()	Wake up a thread that is randomly awakened by wait().
notifyAll();	Wake up all threads that are wait() and re-compete for the time slice



Sync, deadlocks, and how to avoid them



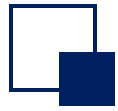
Thread synchronization

- When multiple threads are running at the same time, the scheduling of the threads is determined by the operating system, not by the program itself. Therefore, it is possible for any thread to be suspended by the operating system at any instruction and then resume execution after a certain period of time.
- If multiple threads read and write shared variables at the same time, data inconsistencies will occur.

```
class Counter {  
    public static int count = 0;  
}
```

```
class AddThread extends Thread {  
    public void run() {  
        for (int i=0; i<500; i++) { Counter.count += 1; }  
    }  
}
```

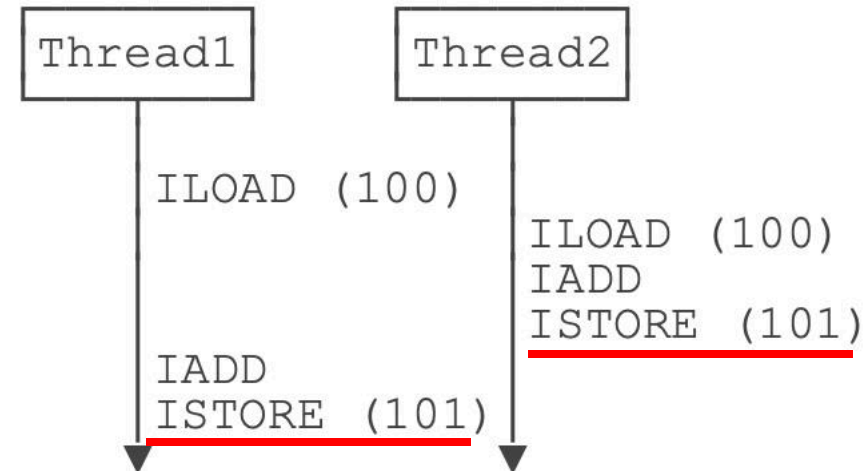
```
class DecThread extends Thread {  
    public void run() {  
        for (int i=0; i<500; i++) { Counter.count -= 1; }  
    }  
}
```

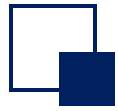


Thread synchronization

- Atomic operation: refers to one or a series of operations that cannot be interrupted. That is, either complete execution or complete non-execution, there is no such thing as half execution.
- Example: $n = n + 1$; It looks like a line of statement, but it actually corresponds to 3 instructions

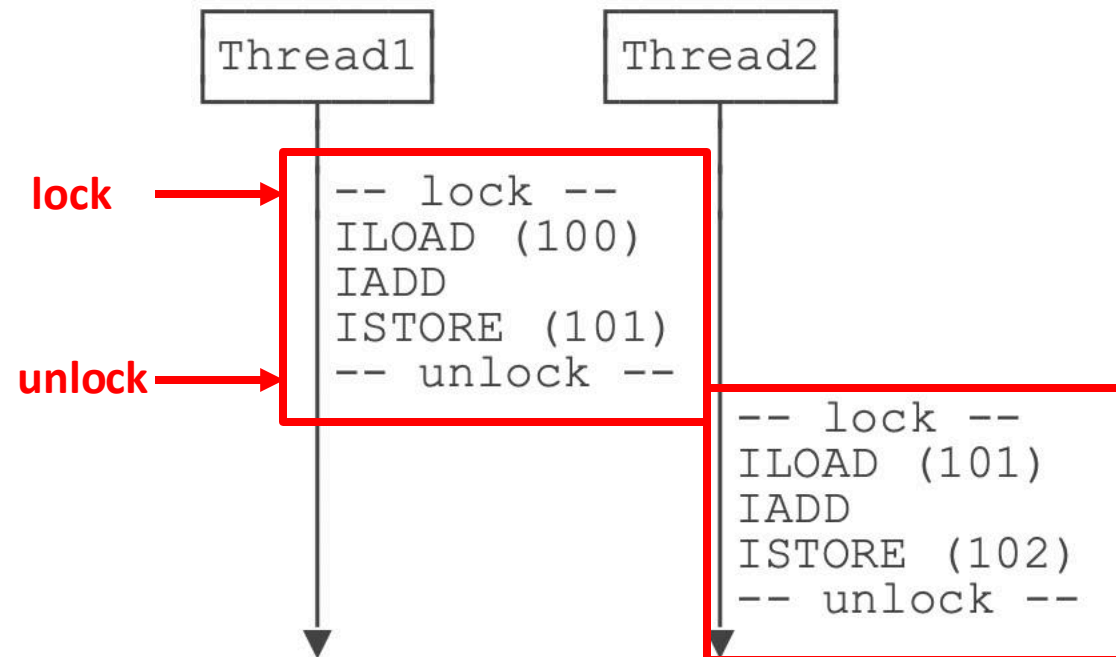
ILOAD	Get the number
IADD	Add
ISTORE	Store outcome





Thread synchronization

- This shows that in the multi-threaded model, in order to ensure that the logic is correct, when reading and writing shared variables, a set of instructions must be executed atomically: that is, when one thread is executed, other threads must wait.



Through the operation of locking and unlocking, it is guaranteed that the 3 instructions will always be executed by one thread, and no other threads will enter this command interval.



Thread synchronization

- Code implementation:

- Java programs use the synchronized keyword to lock an object.

```
synchronized(lock) {  
    n = n + 1;  
}
```

- Rewrite the previous code

```
class AddThread extends Thread {  
    public void run() {  
        for (int i=0; i<10000; i++) {  
            synchronized(Counter.lock) { // Get the lock  
                Counter.count += 1;  
            } // Release the lock  
        }  
    }  
}
```

```
class DecThread extends Thread {  
    public void run() {  
        for (int i=0; i<10000; i++) {  
            synchronized(Counter.lock) { // Get the lock  
                Counter.count -= 1;  
            } // Release the lock  
        }  
    }  
}
```



Thread deadlock

```
class PerA implements Runnable{
    public void run() {
        try {
            System.out.println( "PerA: I have a blue key,
please give me the red key");
            while(true){
                synchronized (DeadLock.bluekey) {
                    System.out.println(" PerA locks the blue key");
                    Thread.sleep(3000); // Waiting here is to give
PerB a chance
                    synchronized (DeadLock.redkey) {
                        System.out.println("PerA takes two keys to
unlock the lock");
                        Thread.sleep(60 * 1000); // for testing
                    }
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
class PerB implements Runnable{
    public void run() {
        try {
            System.out.println("PerB: I have the red key
Please give me the blue key");
            while(true){
                synchronized (DeadLock.redkey) {
                    System.out.println("PerB locks the red key");
                    Thread.sleep(3000); // Waiting here is to give PerA a
chance
                    synchronized (DeadLock.bluekey) {
                        System.out.println("PerB takes two keys to unlock
the lock");
                        Thread.sleep(60 * 1000); // for testing
                    }
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Thread deadlock

- Outcome

PerA: I have a blue key, please give me the red key

PerA locks the blue key

PerB: I have a red key, please give me the blue key

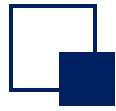
PerB locks the red key

- Deadlock: Thread A (PerA) holds the blue key and wants to get the red key, while thread B (Per) B holds the red key and tries to get the blue key, then these two threads will wait forever.
- Avoid deadlocks: threads acquire locks in a consistent order. That is, strictly follow the order of obtaining the blue key first and then the red key, or the order of obtaining the red key first and then the blue key.



Multithreaded applications

Producer-consumer design pattern

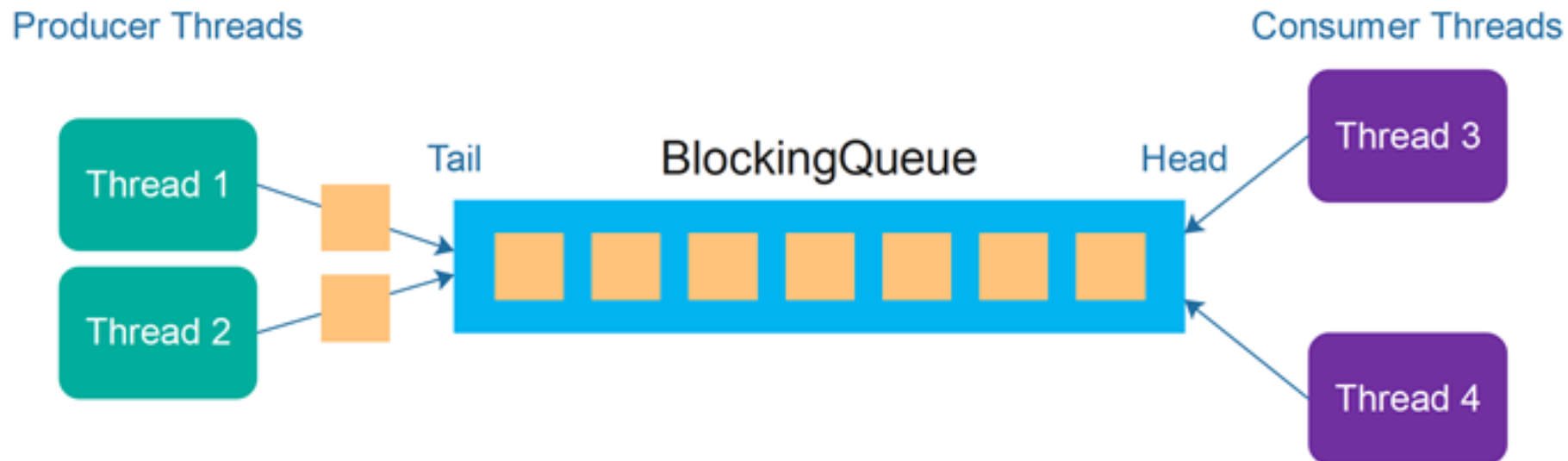


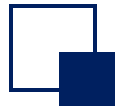
Producer-consumer design pattern

- In the actual software development process, the following scenarios are often encountered:

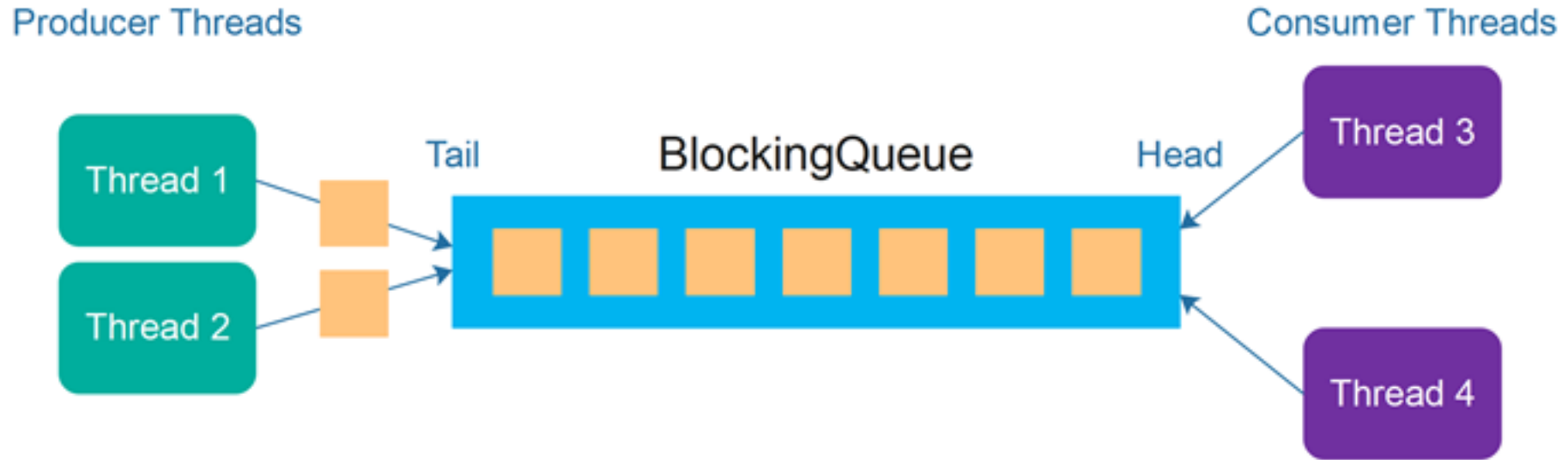
One module is responsible for generating data, which is processed by another module (modules here are generalized, they can be classes, functions, threads, processes, etc.).

- The module that generates data is figuratively called a **producer**; The module that processes data is called a **consumer**.





Producer-consumer design pattern



- If the producer produces too fast, the consumer consumes slowly, and the cache area reaches its maximum. The cache blocks the producer, causing the producer to stop production and wait for the consumer to consume the data before waking up the producer
- When consumers consume too fast, the cache area is empty. The cache blocks consumers and wakes them up after the producer adds data to the queue.

Producer-consumer design pattern

- **Issue:**

- How to ensure the consistency of data state in the cache?
- How to ensure synchronization and collaboration between consumers and producers?

- **Solution:**

- Synchronized
- Utilize direct communication within the thread (Object's wait()/notify() method)

Producer-consumer design pattern

Java code implements buffers

```
public class Buffer {
    private List<Integer> data = new ArrayList<>();
    private static final int MAX = 2;
    private static final int MIN = 0;
    public void put(int value){
        while (true){
            try { //Simulate production data
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized (this){
                //When the container is full, the producer is in a waiting state
                while (data.size() == MAX){
                    System.out.println("buffer is full,waiting ....");
                    try {
                        wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
            //If it is not full, continue to produce
            System.out.println("producer--"+
                Thread.currentThread().getName()+"--put:" + value);
            data.add(value);
            //Wake up all other threads in wait(), including consumers
            and producers
            notifyAll();
        }
    }
}
```


Producer-consumer design pattern

Java code implements buffers

```
public Integer take(){
    Integer val = 0;
    while (true){
        try {//Simulate consumption data
            Thread.sleep(500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        synchronized (this){
            //If there is no data in the container, the consumer is
            in a waiting state
            while (data.size() == MIN){
                System.out.println("buffer is empty,waiting ...");
                try {
                    wait();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

```
//If data is available, continue to consume
    val = data.remove(0) ;
    System.out.println("consumer--"+
Thread.currentThread().getName()+"--take:" + val);
```

```
//Wake up all other threads in wait(), including
consumers and producers
        notifyAll();
    }
}
return val
}
```

Producer-consumer design pattern

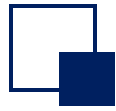
Java code implementation producer

```
public class Producer implements Runnable{  
    private Buffer buffer;  
    public Producer(Buffer buffer) {  
        this.buffer = buffer;  
    }  
    @Override  
    public void run() {  
        buffer.put(new Random().nextInt(100));  
    }  
}
```

Producer-consumer design pattern

Java code implements consumers

```
public class Consumer implements Runnable{  
    private Buffer buffer;  
    public Consumer(Buffer buffer) {  
        this.buffer = buffer;  
    }  
    @Override  
    public void run() {  
        Integer val = buffer.take();  
    }  
}
```



Producer-consumer design pattern

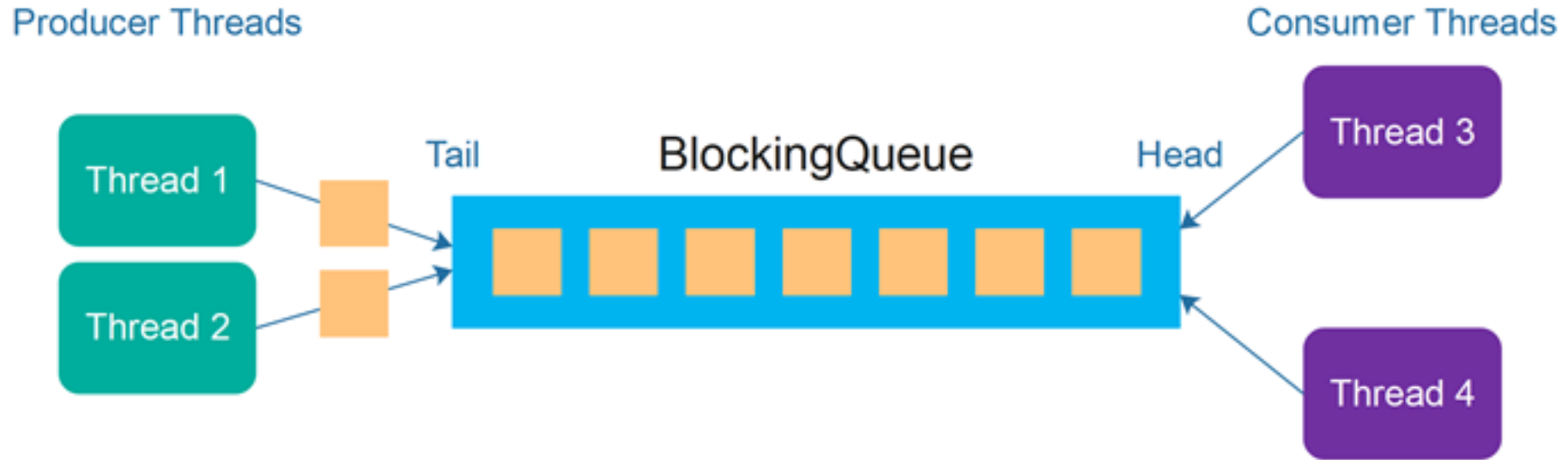
- **Question?**

- Why is the buffer judged by while(condition) instead of if(condition)?
- Why is the wait() method required in Java placed in a sync block?

- **Answer:**

- Preventing threads from being erroneously awakened (Spurious Wake-Up)
- Prevent Lost Wake-Up

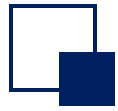
Producer-consumer design pattern



- Advantage:
 - Concurrency (asynchronous): Producers and consumers only need to care about the buffer zone, do not need to pay attention to each other, support high concurrency through asynchronous, and break down a time-consuming process into two stages: production and consumption.
 - Decoupling: Producers and consumers decouple (communicate via buffers).



Task vs. thread pool



Task

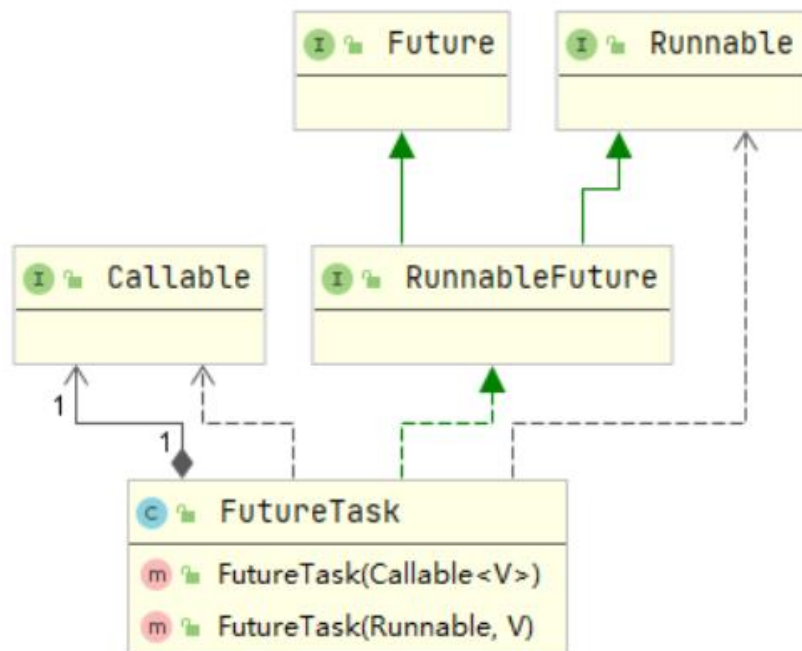
Problem: Expect a run/calculation result to be returned at the end of the thread

- Runnable: Encapsulates a task that runs asynchronously, with no parameters and return values
- Callable: Encapsulates a task that runs asynchronously with a return value
 - call method
 - Callable<Integer> represents an asynchronous computation task that returns an Integer object
 - Exceptions can be thrown
 - You can't thread directly or pass in to Thread

```
public interface Callable<V>{  
    V call throws Exception;  
}
```

FutureTask

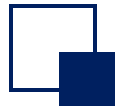
- FutureTask: A method of executing a callable
 - Control the execution of the Callable task and obtain its computational results
 - Indirectly implements the Runnable interface,
 - Threads can then be started through Thread



```
// Instantiate a Callable task with a return type of String
Callable<String> callable = new Callable<String>() {
    public String call() throws Exception {
        // balabalabala~
        return "Hello world ~";
    }
};
// assign callable task to FutureTask
FutureTask<String> task = new FutureTask<String>(callable);

// FutureTask implements the Runnable interface, so it can be
// passed directly to the Thread to start the thread
new Thread(task).start();

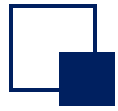
// Get the return value via FutureTask
String call = task.get();
System.out.println(call); // print: Hello world ~
```

FutureTask

- Future: The handle equivalent to the task, used to obtain the result of the task
 - Some methods of obtaining results are defined
 - "Future": Results obtained by calculating in "future"

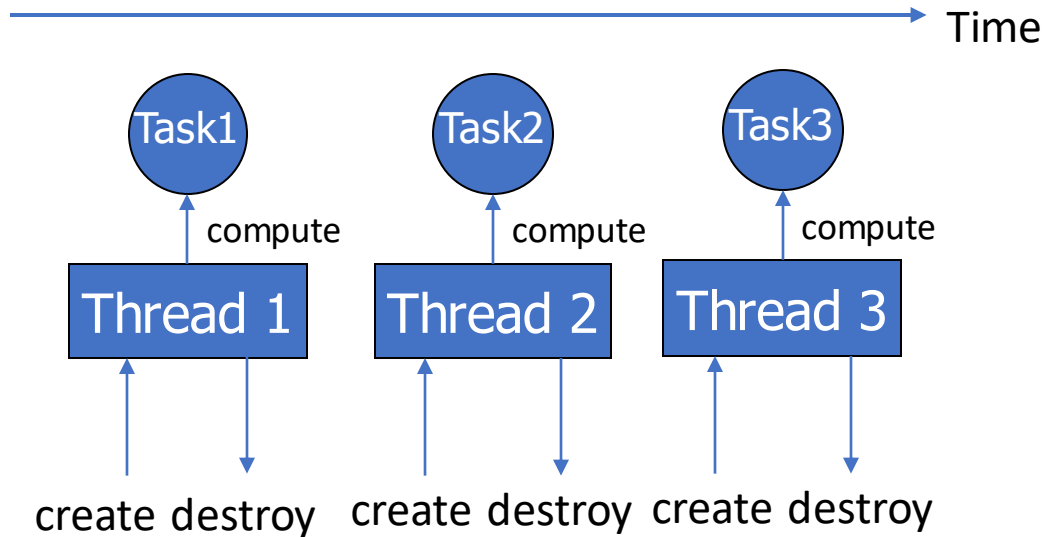
```
public interface Future<V>{  
  
    V get() throws ...;  
    V get(long timeout, TimeUnit unit)  
        throws ...;  
    void cancel(boolean mayInterrupt);  
    boolean isCancelled();  
    boolean isDone();  
  
}
```



Thread pool

Question: What should I do if I create/terminate a large number of threads frequently, which will cause a lot of overhead? **Thread Pool**

Thread pools are not used

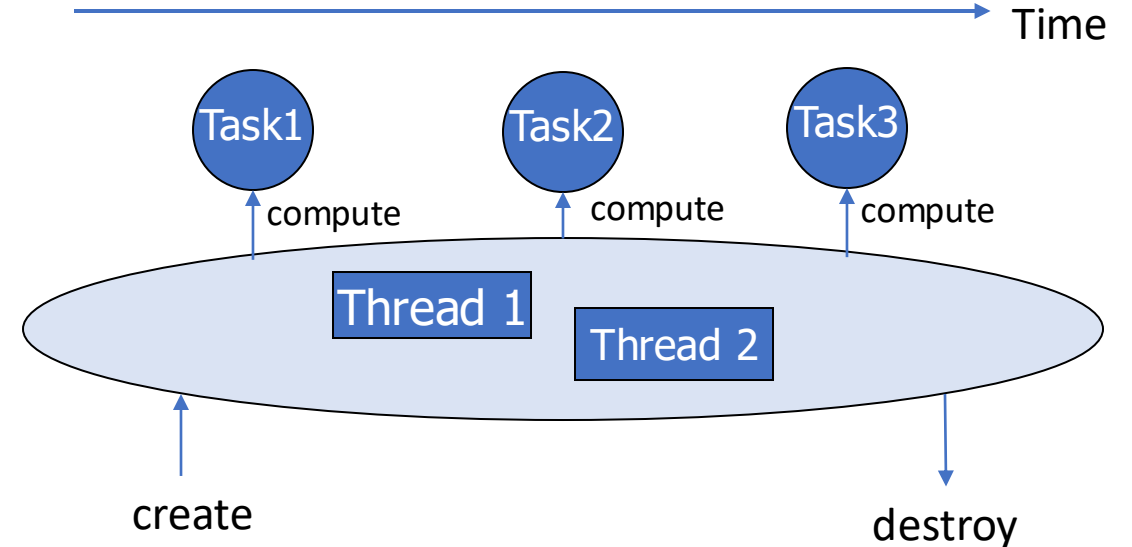


Create a thread for each task, and the thread is destroyed when the task is completed

Analogy: Every time there is work, recruit workers and dismiss workers after work

When there is another job, recruit workers again

Use thread pools



Create threads in the thread pool in advance, and the thread executes the task when the task comes.

After execution, the thread rests and waits for the next task

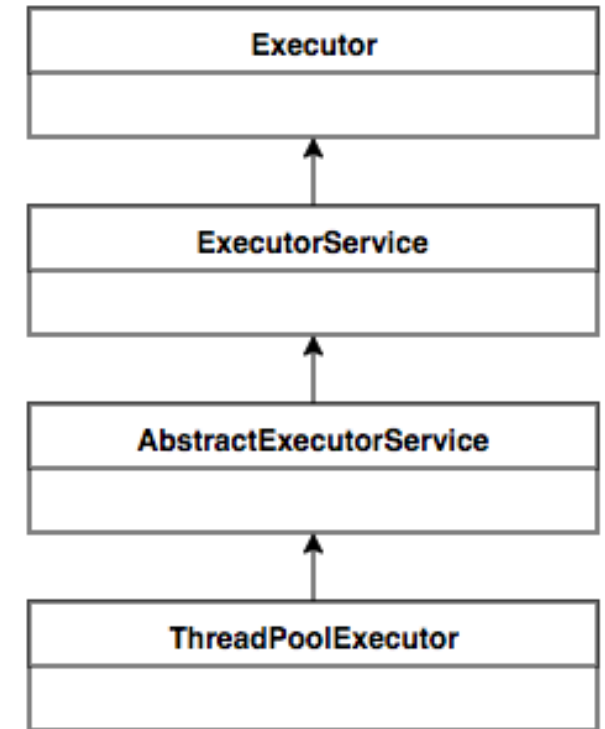
Analogy: The contractor recruits a number of workers in advance, and arranges workers to work when they have work.

Workers rest after completing their work and wait for the next time they are assigned to work

Thread pool

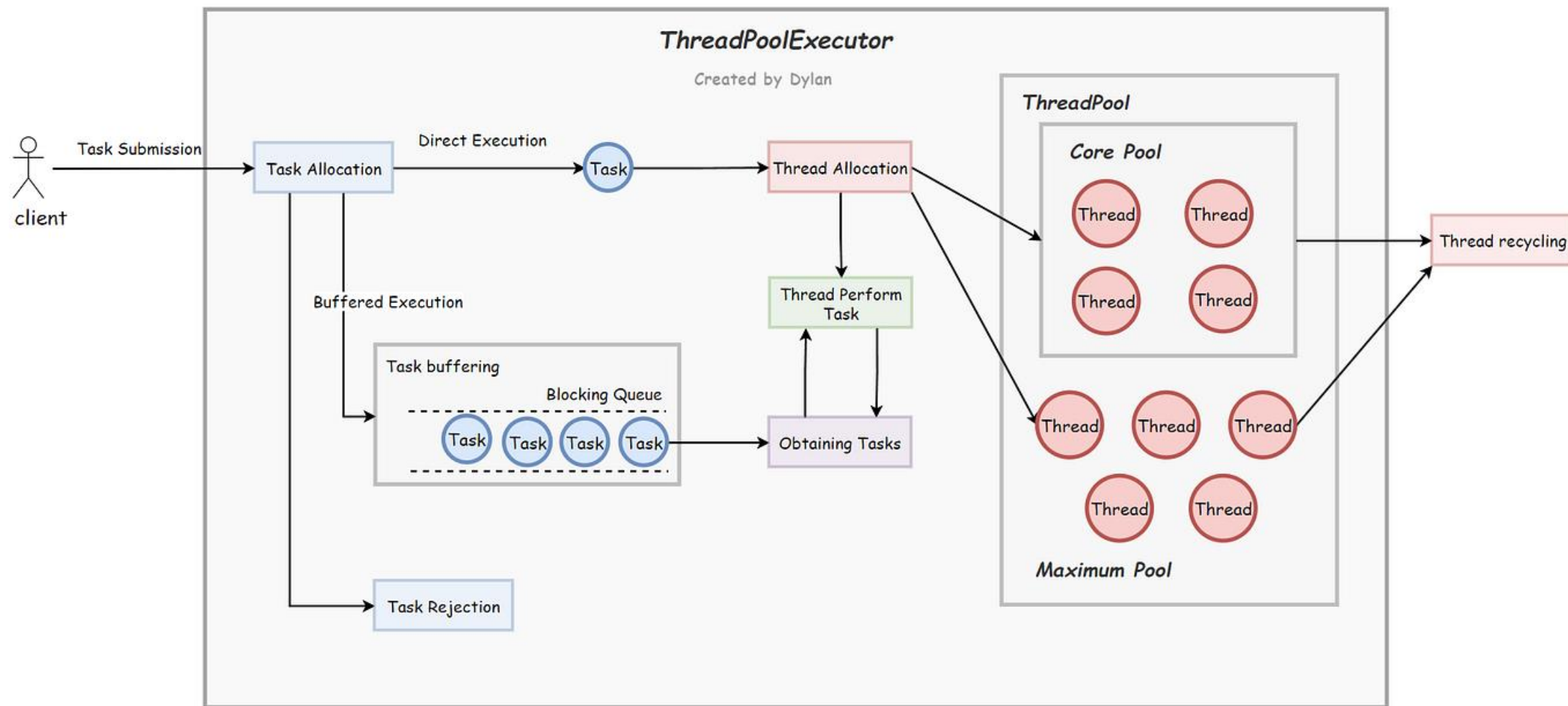
- Inheritance of thread pools

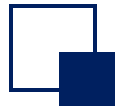
- **Executor**: is an interface that separates business logic commits from task execution
- **ExecutorService**: The interface inherits **Executor** and expands services such as `shutdown()`, which can be said to be a true thread pool interface
- **AbstractExecutorService**: The abstract class implements most of the methods of the **ExecutorService** interface
- **ThreadPoolExecutor**: The core implementation class of the thread pool, which is used to execute the submitted task





Multitasking control for thread pools





Multitasking control for thread pools

- ThreadPoolExecutor thread pool main parameters:

- corePoolSize:

The number of core threads saved in the thread pool

- maximumPoolSize:

The maximum number of threads allowed in the thread pool

- keepAliveTime:

The maximum amount of time that an idle thread can last

- workQueue:

Blocking queues

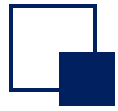
- threadFactory:

The factory where the thread is created, which mainly defines the thread name

- handler:

Rejection policy

```
public ThreadPoolExecutor(  
    int corePoolSize,  
    int maximumPoolSize,  
    long keepAliveTime,  
    TimeUnit unit,  
    BlockingQueue<Runnable> workQueue,  
    ThreadFactory threadFactory,  
    RejectedExecutionHandler handler) {  
}
```



Multitasking control for thread pools

1. First, the thread pool is checked for running status, and if it is not RUNNING, it is directly rejected, and the thread pool must ensure that the task is executed in the RUNNING state.
2. If `workerCount < corePoolSize`, a thread is created and started to execute the newly submitted task.
3. If `workerCount >= corePoolSize` and the blocking queue in the thread pool is not full, the task is added to that blocking queue.
4. If `workerCount >= corePoolSize && workerCount < maximumPoolSize`, and the blocking queue in the thread pool is full, a thread is created and started to execute the newly submitted task.
5. If `workerCount >= maximumPoolSize`, and the blocking queue in the thread pool is full, the task will be handled according to the reject policy, and the default way to handle it is to throw an exception directly.



Question

Problem Description:

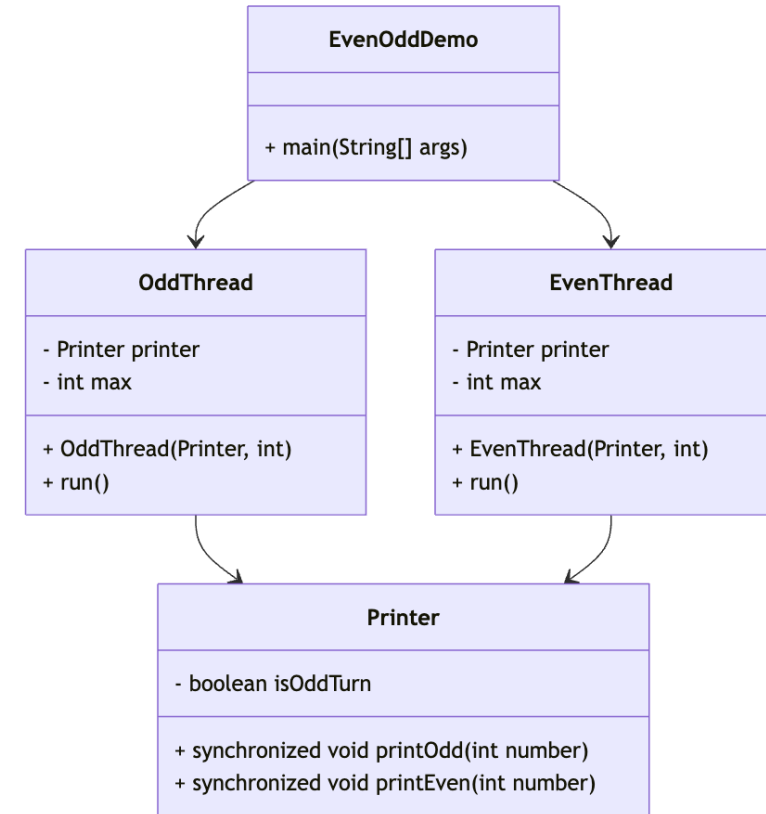
Design a Java program that creates two threads — one thread prints **even numbers** and the other prints **odd numbers** alternately from **1 to 10**.

The threads must coordinate so that the output appears in proper sequence (i.e., 1 2 3 4 5 6 7 8 9 10).

Requirements:

1. Use the Thread class or implement the Runnable interface.
2. Use proper synchronization (e.g., wait() and notify()) to ensure that even and odd numbers print alternately.
3. The main thread should start both threads and wait for them to complete using join().

Expected Output Example: 1 2 3 4 5 6 7 8 9 10





Question

```
class Printer {
    private boolean isOddTurn = true;

    synchronized void printOdd(int number) {
        while (/* TODO: condition to wait when it's not odd's turn */) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        System.out.print(number + " ");
        // TODO: update turn variable
        notify();
    }

    synchronized void printEven(int number) {
        while (/* TODO: condition to wait when it's not even's turn */) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        System.out.print(number + " ");
        // TODO: update turn variable
        notify();
    }
}

class OddThread implements Runnable {
    private final Printer printer;
    private final int max;

    OddThread(Printer printer, int max) {
        this.printer = printer;
        this.max = max;
    }

    public void run() {
        for (int i = 1; i <= max; i += 2) {
            // TODO: call the correct print method
        }
    }
}
```

```
class EvenThread implements Runnable {
    private final Printer printer;
    private final int max;

    EvenThread(Printer printer, int max) {
        this.printer = printer;
        this.max = max;
    }

    public void run() {
        for (int i = 2; i <= max; i += 2) {
            // TODO: call the correct print method
        }
    }
}

public class EvenOddDemo {
    public static void main(String[] args) {
        Printer printer = new Printer();
        Thread oddThread = new Thread(/* TODO: create OddThread instance */);
        Thread evenThread = new Thread(/* TODO: create EvenThread instance */);

        // TODO: start both threads

        try {
            // TODO: wait for both threads to finish
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("\nFinished!");
    }
}
```




Question

```
class Printer {
    private boolean isOddTurn = true;

    synchronized void printOdd(int number) {
        while (!isOddTurn) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        System.out.print(number + " ");
        isOddTurn = false;
        notify();
    }

    synchronized void printEven(int number) {
        while (isOddTurn) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        System.out.print(number + " ");
        isOddTurn = true;
        notify();
    }
}

class OddThread implements Runnable {
    private final Printer printer;
    private final int max;

    OddThread(Printer printer, int max) {
        this.printer = printer;
        this.max = max;
    }

    public void run() {
        for (int i = 1; i <= max; i += 2) {
            printer.printOdd(i);
        }
    }
}
```

```
class EvenThread implements Runnable {
    private final Printer printer;
    private final int max;

    EvenThread(Printer printer, int max) {
        this.printer = printer;
        this.max = max;
    }

    public void run() {
        for (int i = 2; i <= max; i += 2) {
            printer.printEven(i);
        }
    }
}

public class EvenOddDemo {
    public static void main(String[] args) {
        Printer printer = new Printer();
        Thread oddThread = new Thread(new OddThread(printer, 10));
        Thread evenThread = new Thread(new EvenThread(printer, 10));

        oddThread.start();
        evenThread.start();

        try {
            oddThread.join();
            evenThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("\nFinished!");
    }
}
```

Multithreading

- ❑ Processes and threads
- ❑ Multithreading
- ❑ Control of threads in Java
- ❑ Sync, deadlocks, and how to avoid them
- ❑ Multi-threaded applications—producer vs. consumer mode
- ❑ Task vs. thread pool